LAB3

Level 1

```
using UnityEngine;

1 reference
public interface IDamageable
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    0 references
    void Start()
    {

    }

    // Update is called once per frame
    0 references
    void Update()
    {

    }


    //Player Health has access to this class because this class is included in Player Health
    0 references
    void TakeDamage(float amount)
    {


    }
}
```

Created IDamageable Class and created Void Take Damage that receives a float (Floating point Variable)

```
using UnityEngine;

Unity Script | 0 references
public class PlayerHealth : MonoBehaviour  ,IDamageable
{
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    Unity Message | 0 references
    void Start()
    {

    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }
}
```

Created PlayerHealth that has access to the script IDamageable

```
using UnityEngine;

[CreateAssetMenu(fileName = "PlayerStats", menuName = "PlayerStats")]
Unity Script | 0 references
public class PlayerStats : ScriptableObject
{
    [Header("Config")]
    [SerializeField] public int Level;

    [Header("Health")]
    [SerializeField] public int PlayerHealth;
    [SerializeField] public int MaxPlayerHealth;

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    0 references
    void Start()
    {

    }

    // Update is called once per frame
    0 references
    void Update()
    {

    }
}
```

Created a ScriptableObject PlayerStats That will be able to be interacted through unity directly



Locations of all the previous files mentioned in the images

Level 2

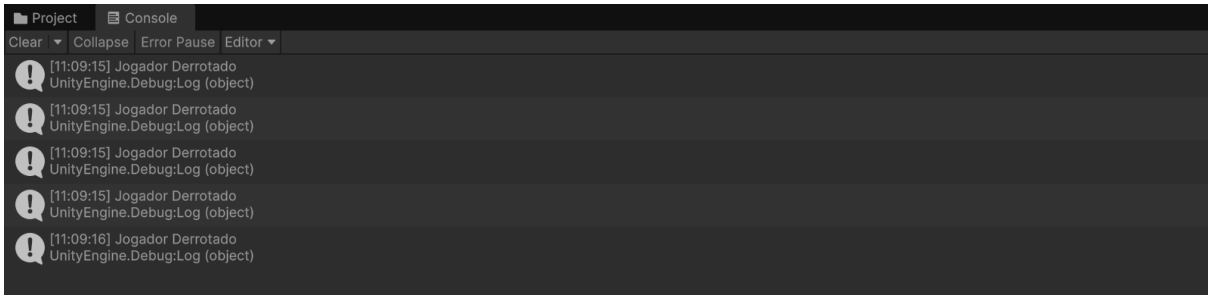
```
2 references
public void TakeDamage(float amount)
{
    stats.PlayerHealth -= amount;

    if (stats.PlayerHealth <= 0)
    {
        PlayerDead();

    }
}

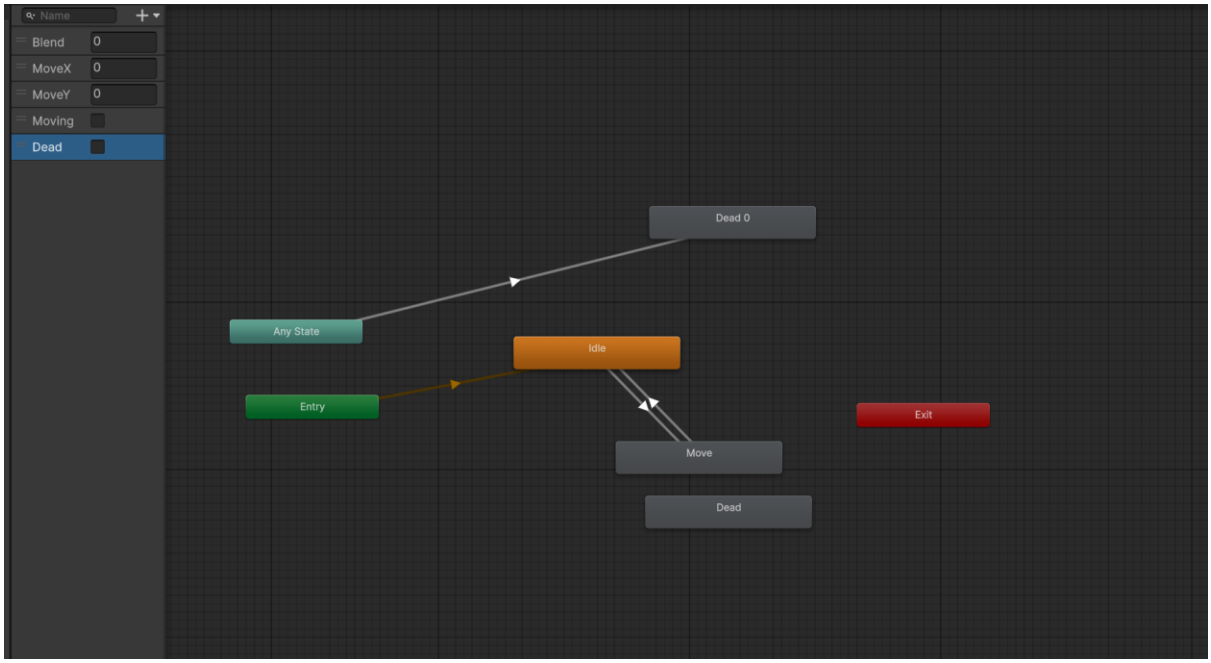
1 reference
public void PlayerDead()
{
    Debug.Log("Jogador Derrotado");
}
```

These pieces of code, make It so when the player takes damage, it reduces their hp by a certain amount, and if that hp goes below 0, theres a message on the log that shows that the player has died



As it can be seen in this image, the player gets a log that reads “Jogador Derrotado”

Level 3



Here is the tree with the added node of the dead state and also parameter that defines a new animation in case the player loses all of its current HP

PlayerStats.cs | PlayerAnimations.cs* | PlayerHealth.cs | IDamageable.cs

Assembly-CSharp | PlayerAnimations

```
using UnityEngine;
using UnityEngine.EventSystems;

Unity Script | 0 references
public class PlayerAnimations : MonoBehaviour
{
    private readonly int moveX = Animator.StringToHash("MoveX");
    private readonly int moveY = Animator.StringToHash("MoveY");
    private readonly int moving = Animator.StringToHash("Moving");
    private readonly int dead = Animator.StringToHash("Dead");

    private Animator animator;

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    Unity Message | 0 references
    void Start()
    {
        animator = GetComponent<Animator>();
    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }

    0 references
    public void SetDeadAnimation()
    {
        animator.SetTrigger(dead);
    }

    0 references
    public void SetMoveBoolTransition(bool Value)
    {


        animator.SetBool(moving, Value);
    }

    0 references
    public void SetMoveAnimation(Vector2 dir)
    {
        animator.SetFloat("moveX", dir.x);
        animator.SetFloat("moveY", dir.y);
    }
}
```

Here is the full PlayerAnimation script that will help set all the different animations

Level 4

```
private void ReadMovement()
{
    moveDirection = actions.Moveset.Move.ReadValue<Vector2>().normalized;
    if (moveDirection == Vector2.zero)
    {
        anim.SetMoveBoolTransition(false);

        return;
    }
    anim.SetMoveBoolTransition(true);
    anim.SetMoveAnimation(moveDirection);

}
```

In this code, we changed how the movement animations in the game worked by making the code simpler, so instead of checking for vertical and sideways movement, it only checks for movement and then attributes the animation to where the character is moving

```
2 references
public void TakeDamage(float amount)
{
    stats.PlayerHealth -= amount;

    if (stats.PlayerHealth <= 0)
    {
        PlayerDead();

    }

}

1 reference
public void PlayerDead()
{
    Debug.Log("Jogador Derrotado");
    health_anim.SetDeadAnimation();
}
```

And finally, we added death animations, by which mainly just checks if the player's health has gone to 0 or lower, and then making it play the death animation

Level 5

```
using UnityEngine;

Unity Script | 0 references
public class Player : MonoBehaviour
{

    [SerializeField] private PlayerStats stats_player;
    0 references
    public PlayerStats Stats => stats_player;

    // Start is called once before the first execution of Update after the MonoBehaviour is created
    Unity Message | 0 references
    void Start()
    {

    }

    // Update is called once per frame
    Unity Message | 0 references
    void Update()
    {

    }
}
```

Here we are starting the progress of making the player not move when they die, which mainly consists of making a private variable available for other classes to use

```
[Header("Config")]
[SerializeField] private float speed;
private Player_actions actions;
private Rigidbody2D rigid_var;
private Vector2 moveDirection;
private Animator anim_ninja;
private PlayerAnimations anim;
private Player player_move;

Unity Message | 0 references
void Awake()
{
    rigid_var = GetComponent<Rigidbody2D>();
    actions = new Player_actions();
    anim_ninja = GetComponent<Animator>();
    anim = GetComponent<PlayerAnimations>();
    player_move = GetComponent<Player>();
}
```

We added a variable to PlayerMovement, with access to the previously made class

```
private void Move()
{
    if(player_hand.Stats.PlayerHealth <= 0) {

        return;

    }

        rigid_var.MovePosition(rigid_var.position + moveDirection * (speed * Time.fixedDeltaTime));
}
```

And finally, finishing what was started last image, we have it so if the player's health goes to 0 or below, they stop being able to move